

Compte rendu de projet S3/S4

Développement d'un capteur environnemental

Membres : Raphaël LONGUET, Pierrick CLEMENT, Louan CAUMETTE, Tymeo CHALON Formation : BUT Mesures Physiques Année universitaire : BUT2 (2025-2026) Établissement IUT Mesures Physiques - Université de Toulouse Encadrants : Mme BRUT & Mr DEZALAY	
--	--

Réalisé dans le cadre d'un projet d'étude

1. Sommaire

Contents

1. Sommaire	2
2. Résumé	3
3. Introduction	3
4. Principe de la reflectance et du NDVI	3
4.1. La Réflectance Végétale	3
4.2. L'Indice NDVI	4
5. Cahier des charges	4
5.1. Fonctions principales	4
5.2. Contraintes techniques	4
5.3. Architecture du système	4
5.4. Acquisition et gestion des données	5
5.5. Contraintes d'exploitation	5
6. Choix des capteurs et inventaire	5
6.1. Éléments obligatoires	5
6.2. Éléments fortement recommandés ($\approx 90\%$)	6
6.3. Éléments optionnels ($\approx 70\%$)	6
6.4. Intégration et fabrication	6
7. Architecture matérielle	6
8. Architecture logicielle	7
8.1. Organisation des programmes	7
9. Tests et validation	8
9.1. Tests	8
9.2. Résultats	9
10. Conclusion	10
11. Annexe	11

2. Résumé

Ce projet d'étude avait pour objectif de concevoir et de réaliser un capteur environnemental fonctionnel destiné au suivi de l'état de santé de la vigne. Le principe repose sur l'analyse de la réflectance d'une feuille de vigne afin de calculer l'indice de végétation NDVI, indicateur largement utilisé pour évaluer l'état physiologique des plantes.

Le système développé s'appuie sur deux capteurs de réflectance multispectraux positionnés autour de la feuille, permettant une mesure plus représentative du flux lumineux réfléchi. Ces capteurs sont pilotés par un microcontrôleur, qui assure l'acquisition des données, leur horodatage ainsi que leur stockage sur carte SD. Des capteurs de température et d'humidité ont également été intégrés afin de contextualiser les mesures de réflectance. Une attention particulière a été portée à la gestion de l'énergie et à la protection du système pour garantir un fonctionnement autonome et fiable en conditions extérieures.

Ce projet a permis de mettre en œuvre une chaîne de mesure complète, depuis le choix des capteurs jusqu'à l'exploitation des données, tout en répondant à une problématique concrète liée à l'agriculture et à la viticulture de précision.

3. Introduction

La surveillance de l'état de santé des cultures constitue un enjeu majeur en agriculture, notamment dans le domaine viticole où la qualité et le rendement de la vigne dépendent fortement de facteurs environnementaux et physiologiques. Parmi les méthodes de suivi existantes, l'analyse de la réflectance des feuilles apparaît comme un outil pertinent pour évaluer l'état sanitaire et le niveau de stress des plantes.

Dans ce contexte, ce projet d'étude a pour objectif de développer un capteur environnemental fonctionnel et "low-cost" capable d'étudier la réflectance d'une feuille de vigne. Le principe repose sur la mesure de la lumière du soleil réfléchie par la feuille, ces informations étant directement liées à des paramètres tels que la teneur en chlorophylle, l'hydratation ou la présence de maladies.

Le capteur développé devra être en mesure d'acquérir des données fiables, reproductibles et exploitables, tout en respectant des contraintes de simplicité, de coût et de consommation énergétique. Ce projet permet ainsi de mettre en œuvre des compétences en capteurs optiques, acquisition de données et conception d'algorithme, tout en répondant à une problématique concrète liée au suivi de la santé de la vigne.

4. Principe de la réflectance et du NDVI

4.1. La Réflectance Végétale

La réflectance est la capacité d'une surface à renvoyer la lumière. Les plantes absorbent le rayonnement solaire pour la photosynthèse et réfléchissent certaines longueurs d'onde. Une feuille saine absorbe beaucoup de lumière rouge et réfléchit fortement le proche infrarouge (NIR).

4.2. L'Indice NDVI

Le Normalized Difference Vegetation Index (NDVI) quantifie la santé de la végétation. Il est calculé à partir des mesures de réflectance dans le rouge et le proche infrarouge. Un NDVI élevé indique une végétation dense et saine

On le calcule grace aux formules suivantes :

$$NDVI = \frac{\rho NIR - \rho R}{\rho NIR + \rho R}$$

$$NDVI = \frac{\frac{\rho NIR_{down}}{\rho NIR_{up}} - \frac{\rho R_{down}}{\rho R_{up}}}{\frac{\rho NIR_{down}}{\rho NIR_{up}} + \frac{\rho R_{down}}{\rho R_{up}}}$$

5. Cahier des charges

Ce projet vise à concevoir un capteur de réflectance environnemental destiné au suivi de l'état de santé de la vigne par le calcul de l'indice de végétation NDVI (Normalized Difference Vegetation Index). Le système doit permettre une exploitation fiable et durable des mesures en conditions réelles.

5.1. Fonctions principales

- Mesurer la réflectance de la feuille de vigne sur deux faces (capteur de réflectance double face : une face qui vise le soleil et un autre qui vise la feuille).
- Calculer le NDVI à partir des données acquises.
- Mesurer la température et l'humidité relative de l'environnement afin de contextualiser les mesures de réflectance.
- Assurer l'acquisition, le stockage et l'exploitation des données de manière autonome.

5.2. Contraintes techniques

- Le capteur de réflectance qui vise la feuille doit être positionné au-dessus de la feuille, avec un angle d'incidence adapté afin d'optimiser la qualité de la mesure.
- Un diffuseur doit être ajouté sur le capteur visant le Soleil afin d'assurer une répartition homogène du flux lumineux mesuré.
- Le système doit être protégé mécaniquement et isolé afin de résister aux conditions environnementales extérieures (humidité, poussière, variations de température).
- L'ensemble capteur – unité de traitement doit présenter une bonne stabilité et une reproductibilité satisfaisante des mesures.

5.3. Architecture du système

- Capteur de réflectance optique double face.
- Capteur de température/humidité relative.
- Microcontrôleur (Arduino).
- Système de stockage des données sur carte SD.
- Système d'alimentation adapté au fonctionnement autonome du dispositif.
- Optionnellement un module LORA.

5.4. Acquisition et gestion des données

- Les données sont acquises par le microcontrôleur Arduino.
- Les mesures sont réalisées toutes les 15 minutes.
- À chaque cycle, l'acquisition est effectuée pendant une durée d'une minute.
- Les valeurs enregistrées correspondent à la moyenne des mesures effectuées durant cette minute.
- Les données sont stockées sur une carte SD sous un format exploitable pour une analyse ultérieure.
- Le module LORA pourrait éventuellement être utilisé afin de pouvoir transmettre les données à distance.

5.5. Contraintes d'exploitation

- Le capteur doit être simple à mettre en œuvre sur le terrain.
- Le coût global du système doit rester compatible avec une application pédagogique et expérimentale.
- Le dispositif doit permettre une utilisation prolongée sans intervention fréquente.

6. Choix des capteurs et inventaire

Cette section présente les différents composants matériels retenus pour la réalisation du capteur environnemental, ainsi que leur rôle au sein du système. Les éléments sont classés selon leur niveau de nécessité pour le fonctionnement global du dispositif.

6.1. Éléments obligatoires

Les composants suivants sont indispensables au bon fonctionnement du capteur et à la réalisation des objectifs du projet.

- **Capteurs de réflectance (×2) : Spectral Sensor Breakout – AS7263 NIR.** Deux capteurs sont utilisés afin de mesurer pour l'un, la réflectance sur la face de la feuille de vigne exposée au soleil, le deuxième capteur mesure la luminosité du soleil. Ce capteur multispectral proche infrarouge est particulièrement adapté au calcul du NDVI.
- **Diffuseurs optiques :** placés au-dessus des capteurs de réflectance, ils assurent une répartition homogène du flux lumineux incident et réfléchi, améliorant la répétabilité des mesures.
- **Microcontrôleur : Arduino Nano 33 IoT.** Il assure le pilotage des capteurs, l'acquisition des données, leur traitement et leur stockage. Ce module a été retenu pour sa compacité, sa faible consommation et ses capacités de communication, et surtout ses sorties logiques en 3.3V.
- **Multiplexeur I2C : TCA9548A.** Il permet de gérer plusieurs capteurs sur un même bus I2C tout en évitant les conflits d'adressage.
- **Module de carte SD : ARD Module SD.** Il est utilisé pour le stockage local des données de mesure, permettant leur récupération et leur analyse ultérieure.
- **Système d'alimentation :** alimentation par batterie, basée sur des piles AA ou des batteries Li-ion de type 18650, le choix final dépendant de la consommation globale du système.
- **Capteur de température et d'humidité relative : BME280.** Ce capteur permet de mesurer la température et l'humidité ambiante afin de contextualiser les mesures de réflectance.
- **Horloge temps réel (RTC) : DS1307.** Elle assure l'horodatage précis des données enregistrées sur la carte SD.
- **Low-Power Timer : TPL5110.** Ce composant est utilisé pour gérer les cycles de veille et d'acquisition, contribuant significativement à la réduction de la consommation énergétique.

- **Éléments d'étanchéité** : joints toriques et sachets anti-humidité, destinés à protéger l'électronique contre l'humidité et la condensation en conditions extérieures.

6.2. Éléments fortement recommandés ($\approx 90\%$)

Ces composants améliorent la fiabilité et la durabilité du système.

- **Radiateur passif** : utilisé pour limiter l'échauffement des composants électroniques et réduire les dérives thermiques susceptibles d'affecter les mesures.

6.3. Éléments optionnels ($\approx 70\%$)

Les éléments suivants apportent des fonctionnalités supplémentaires au système.

- **Module de communication LoRa : M029-V11**. Il permet la transmission des données à distance, limitant la nécessité d'une récupération manuelle de la carte SD.
- **Accéléromètre** : permet la détection de mouvements, de chocs ou d'un déplacement non souhaité du capteur.
- **Panneau solaire** : utilisé pour augmenter l'autonomie énergétique du dispositif.

6.4. Intégration et fabrication

- **Impression 3D** : utilisée pour la conception du boîtier et des supports mécaniques, permettant une intégration sur mesure du système.
- **Matériaux d'étanchéité** : sélectionnés afin de garantir la résistance du capteur aux conditions environnementales extérieures.

7. Architecture matérielle

Le système fonctionne de la façon suivante :

• Prise de mesures / Acquisition de données

Les deux capteurs de réflectance mesurent la réflectance. Le premier vise le soleil (vise à 90° par rapport au sol) pour profiter de l'exposition du soleil la plus optimale. Le deuxième capteur est orienté sur la feuille de vigne avec un certain angle pour que le capteur ne face pas d'ombre sur la feuille, ce qui pourrait fausser la mesure.

Le BME 280 effectue de son côté la mesure de la température ambiante ainsi que l'humidité autour de l'installation afin de contextualiser le milieu dans lequel la prise de mesures est effectuée.

• Stockage des données

Les données mesurées sont transférées sur une carte SD qui enregistre les mesures ce qui permet de conserver les mesures avant de les transmettre.

• Transmission des données

Les données de la carte SD sont transmises grâce au module LoRa qui permet de mener les mesures à distance.

• Interface et traitement des données

Les données transmises arrivent dans un logiciel conçu pour traiter les grandeurs mesurées, calculer le NDVI et horodater les mesures, puis exprimer les résultats sous forme de graphiques temporels pour avoir un suivi temporel des mesures.

Un historique est aussi intégré dans l'interface pour que l'utilisateur puisse être en mesure de retrouver une mesure affectée de manière précise (heure/jour/mois/année)

- **Résumé de l'architecture matérielle du capteur :**



Figure 1: schéma bloc du fonctionnement du capteur

8. Architecture logicielle

Le logiciel embarqué assure l'acquisition des mesures, leur traitement et leur transmission.

Nous avons décidé de séparer le code en 2 parties : une partie en C, sur Arduino IDE, qui contrôle l'Arduino directement et une autre sur Python, qui gère l'interface graphique. De plus, nous avons transformé ce même code python en programme exécutable (.exe) pour avoir au final une application fonctionnelle, déposée dans la SD. Cette appli permet de charger le fichier .csv donné directement en sortie du code Arduino, et lui même posé dans la SD.

Le but au final est d'avoir sur la SD :

- Le fichier des mesures, du lancement jusqu'à l'arrêt.
- L'application pour lire ce même fichier.

8.1. Organisation des programmes

1) Programme Arduino

Le programme Arduino est très simple:

On définit les bibliothèques à utiliser (1 par capteur).

On initialise les capteurs (1 initialisation par capteur, pour vérifier la communication entre Arduino-Capteurs)

On prends les mesures sur les différents capteurs, puis on les communique à la SD, qui les dépose dans le fichier .csv !

Le code complet est mis en annexe.

2) Programme Python + Programme exécutable

C'est ici que le code devient compliqué. Pour ne pas mentir, nous ne pouvions pas, avec nos compétences seules, réaliser ce programme. Certaines parties du programmes sont récupérées directement de "sites librairies" et même générés par IA. Le but pour nous était de compiler le tout, comprendre le programme en fond, et y mettre la forme.

Le programme est séparé en plusieurs parties :

- La première consiste à créer une page séparée de l'interface Python créant les onglets voulus : une Dashboard pour voir les différentes valeurs moyennes des capteurs, des pages avec des graphiques, et une page avec l'historique des mesures.

-La deuxième partie comprends tout un tas de fonctions utiles : une fonction qui ajoute un bouton pour charger le fichier CSV qui sort de l'Arduino, trois fonctions qui créent les tableaux et graphiques sur les différentes pages et enfin deux fonctions qui créent les fameux boutons qui permettent de charger un jour en particulier ou une heure en particulier.

Le code est aussi disponible en annexe, bien sûr il est annoté !

9. Tests et validation

De nombreux tests ont été réalisés afin de valider le bon fonctionnement du capteur et de s'assurer du bon fonctionnement du système.

9.1. Tests

- Dans un premier temps nous avons effectués des tests sur chaque composant pour s'assurer du bon fonctionnement de chacun d'entre eux, puis nous avons commencés à relier les différents composants entre eux afin de tester qu'ils fonctionnent ensemble.
- En même temps que nous étions en train de développer l'interface d'acquisition, nous avons entamés les premiers tests sur le module LoRa pour commencer les premières transmissions à distance.
- Nous avons ensuite conçu le PCB sur le logiciel EAGLE et avons commandé sa production en vue de l'assembler à l'avenir.
- Le boîtier d'acquisition sur Fusion 360 a été par la suite développé pour placer stratégiquement les capteurs tout en s'assurant du bon fonctionnement du système.
- Au final nous avons réussi à développer un capteur capable de :
 - Prendre des mesures (Reflectance, Température, Humidité)
 - Stocker ces mesures dans une carte SD
 - Les envoyer à distance vers un programme exécutable capable de traiter et d'exprimer correctement ces mesures à l'utilisateur.

9.2. Résultats

Présentation et analyse des résultats obtenus.

10. Conclusion

Le développement de ce capteur environnemental a permis de répondre aux objectifs fixés dans le cahier des charges, en proposant une solution capable de mesurer la réflectance d'une feuille de vigne et d'exploiter ces données pour le calcul du NDVI. Le système conçu est fonctionnel, malgré un certain retard que ce soit par manque d'expérience, d'accidents et de retards (notamment pour les composants) nous sommes parvenus à fournir une chaîne de mesures qui marche avec un potentiel que nous allons améliorer.

Ce travail a mis en évidence l'importance du choix des capteurs, de leur intégration mécanique et de la gestion énergétique dans la fiabilité des mesures. Les différentes phases du projet, allant de la conception à la validation, ont permis d'appliquer de manière concrète les notions de capteurs, d'acquisition de données et de traitement du signal.

Des améliorations peuvent néanmoins être envisagées, notamment la transmission à distance pour éviter la dégradation du LoRa, l'optimisation de la consommation énergétique ou encore l'intégration d'une alimentation solaire. Ce projet constitue ainsi une base solide pour le développement futur de systèmes de suivi de la santé des cultures dans le cadre de l'agriculture de précision.

- Programmes:

```
#####
# BILIBIOTHEQUES #####
#include <Wire.h> // Communication I2C
#include <SPI.h> // Communication SPI (SD + LoRa)
#include <SD.h> // Gestion de la carte SD
#include <RTClib.h> // Gestion de la RTC DS1307
#include <AS726X.h> // Capteur spectral AS726X
#include <Adafruit_BME280.h> // Capteur BME280 Temp/hum/press

// =====
// CONSTANTES / PINOUT
// =====
const int chipSelect = 18; // CS SD
const uint16_t TCA_ADDR = 0x76; // Adresse I2C par défaut du multiplexeur

// =====
// INSTANCES
// =====
AS726X sensor1; // Capteur AS726X #1
Adafruit_BME280 bme; // Capteur BME280
RTC_DS1307 rtc; // RTC

// Buffers date/heure
char datebuff[11];
char timebuff[9];

// =====
// FONCTION UTILITAIRE : choisir un canal du TCAD45A
// =====
void selectCanal(uint8_t channel) {
    if (channel >= 3) return; // 4 canaux : 0 à 3
    Wire.beginTransmission(TCA_ADDR);
    Wire.write(channel); // Sélectionne le canal voulu
    Wire.endTransmission();
}

// =====
// SETUP
// =====
void setup() {
    Serial.begin(115200); // Attente port série
    Wire.begin();

    // --- INIT Multiplexeur + AS726X #1 --- Canal 0
    selectCanal(0);
    if (!sensor1.begin()) {
        Serial.println("Erreur AS726X #1 !");
        while(1);
    }

    // --- INIT Multiplexeur + AS726X #2 --- Canal 1
    selectCanal(1);
    if (!sensor1.begin()) {
        Serial.println("Erreur AS726X #2 !");
        while(1);
    }

    // --- INIT BME280 ---
    if (!bme.begin(0x76)) {
        Serial.println("Erreur BME280 !");
        while(1);
    }

    // --- INIT RTC ---
    if (!rtc.begin()) {
        Serial.println("RTC non détectée !");
        while(1);
    }
    if (!rtc.isrunning()) {
        rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
    }

    // --- INIT SD ---
    if (!SD.begin(chipSelect)) {
        Serial.println("Erreur SD !");
        while(1);
    }
    if (!SD.exists("Resumes.csv")) {
        File dataFile = SD.open("Resumes.csv", FILE_WRITE);
        if (dataFile) {}
    }

    dataFile.close();

    Serial.println("\n Setup terminé !");

    // =====
    // LOOP : Lecture Captures
    // =====
    void loop() {
        // --- Date/Heure ---
        DateTime now = rtc.now();
        sprintf(datebuff, "%d-%d-%d", now.year(), now.month(), now.day());
        sprintf(timebuff, "%d-%d-%d", now.hour(), now.minute(), now.second());

        // --- Lecture AS726X #1 ---
        selectCanal(0);
        sensor1.readRawData(); // Sélection canal 0
        float R1 = sensor1.getAlbedoRead();
        float G1 = sensor1.getAlbedoRead();
        float B1 = sensor1.getAlbedoRead();
        float V1 = sensor1.getAlbedoRead();
        float W1 = sensor1.getAlbedoRead();
        float tempM1 = sensor1.getTemperature();

        // --- Lecture AS726X #2 ---
        selectCanal(1);
        sensor1.readRawData(); // Sélection canal 1
        float R2 = sensor1.getAlbedoRead();
        float G2 = sensor1.getAlbedoRead();
        float B2 = sensor1.getAlbedoRead();
        float V2 = sensor1.getAlbedoRead();
        float W2 = sensor1.getAlbedoRead();
        float tempM2 = sensor1.getTemperature();

        // --- Lecture BME280 ---
        float tempDPE = bme.readTemperature();
        float humidD = bme.readHumidity();
        float pressDPE = bme.readPressure() / 100.0;

        // NDVI
        float NDVI = 0.0; // Valeur par défaut

        // Vérifie que les dénominateurs ne sont pas nuls
        if ((V2 != 0.66 || V2 == 0.66) / (V1 + V2) != 0) {
            NDVI = (R1 - V2) / ((R1 + V2) / 2 + ((V1 + V2) / 2));
        }

        // Stockage SD ---
        File dataFile = SD.open("Resumes.csv", FILE_WRITE);
        if (dataFile) {
            dataFile.print(datebuff); dataFile.print(",");
            dataFile.print(timebuff); dataFile.print(",");

            // AS726X #1
            dataFile.print(R1); dataFile.print(",");
            dataFile.print(G1); dataFile.print(",");
            dataFile.print(B1); dataFile.print(",");
            dataFile.print(V1); dataFile.print(",");
            dataFile.print(W1); dataFile.print(",");
            dataFile.print(tempM1); dataFile.print(",");

            // AS726X #2
            dataFile.print(R2); dataFile.print(",");
            dataFile.print(G2); dataFile.print(",");
            dataFile.print(B2); dataFile.print(",");
            dataFile.print(V2); dataFile.print(",");
            dataFile.print(W2); dataFile.print(",");
            dataFile.print(tempM2); dataFile.print(",");

            // Temperature AS
            dataFile.print(tempDPE); dataFile.print(",");
            dataFile.print(humidD); dataFile.print(",");

            // BME280
            dataFile.print(tempDPE); dataFile.print(",");
            dataFile.print(humidD); dataFile.print(",");
            dataFile.print(pressDPE); dataFile.print(",");

            // NDVI
            dataFile.print(NDVI);

            dataFile.close();
        }

        // --- Affichage Serial
        Serial.print(datebuff); Serial.print(" "); Serial.print(timebuff);
        Serial.print("ASI NDVI: "); Serial.print(NDVI); Serial.print("\n");

        delay(2000); // Lecture toutes les 2s (ajustable)
    }
}
```

Figure 2: Code arduino, controle du capteur NVDI

